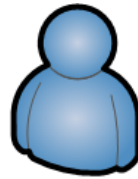


Consistency and CAP

Reasons for Replication

- Data replication is a common technique in distributed systems. There are two reasons for data replication:
 - It increases the **reliability** of a system.
 - If one replica is unavailable or crashes, use another
 - Protect against corrupted data
 - It improves the **performance** of a system. 性能
 - Scale with size of the distributed system (replicated Web servers)
 - Scale in geographically distributed systems (Web proxies)
- The key issue is the need to maintain **consistency** of replicated data.
 - If one copy is modified, others become inconsistent.

Potential Issue



$X = 0$

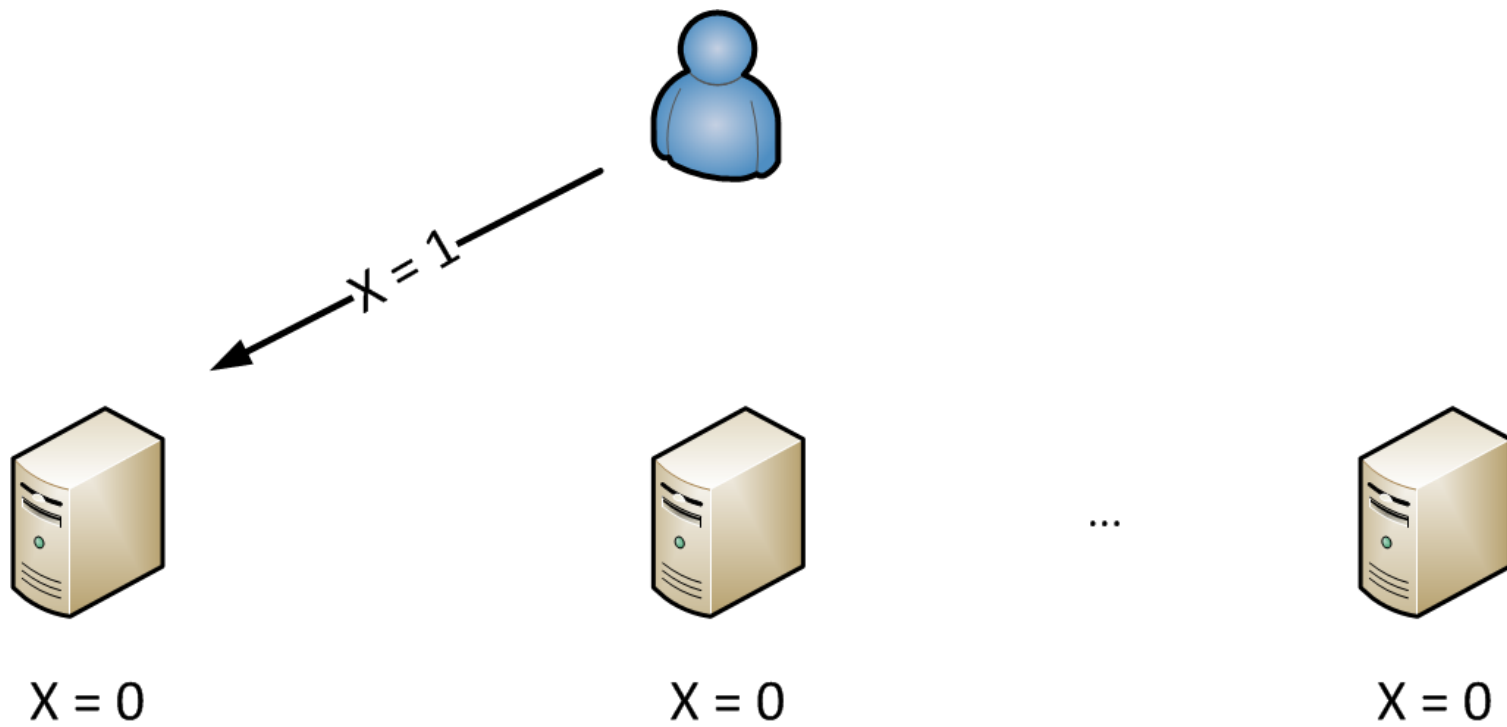


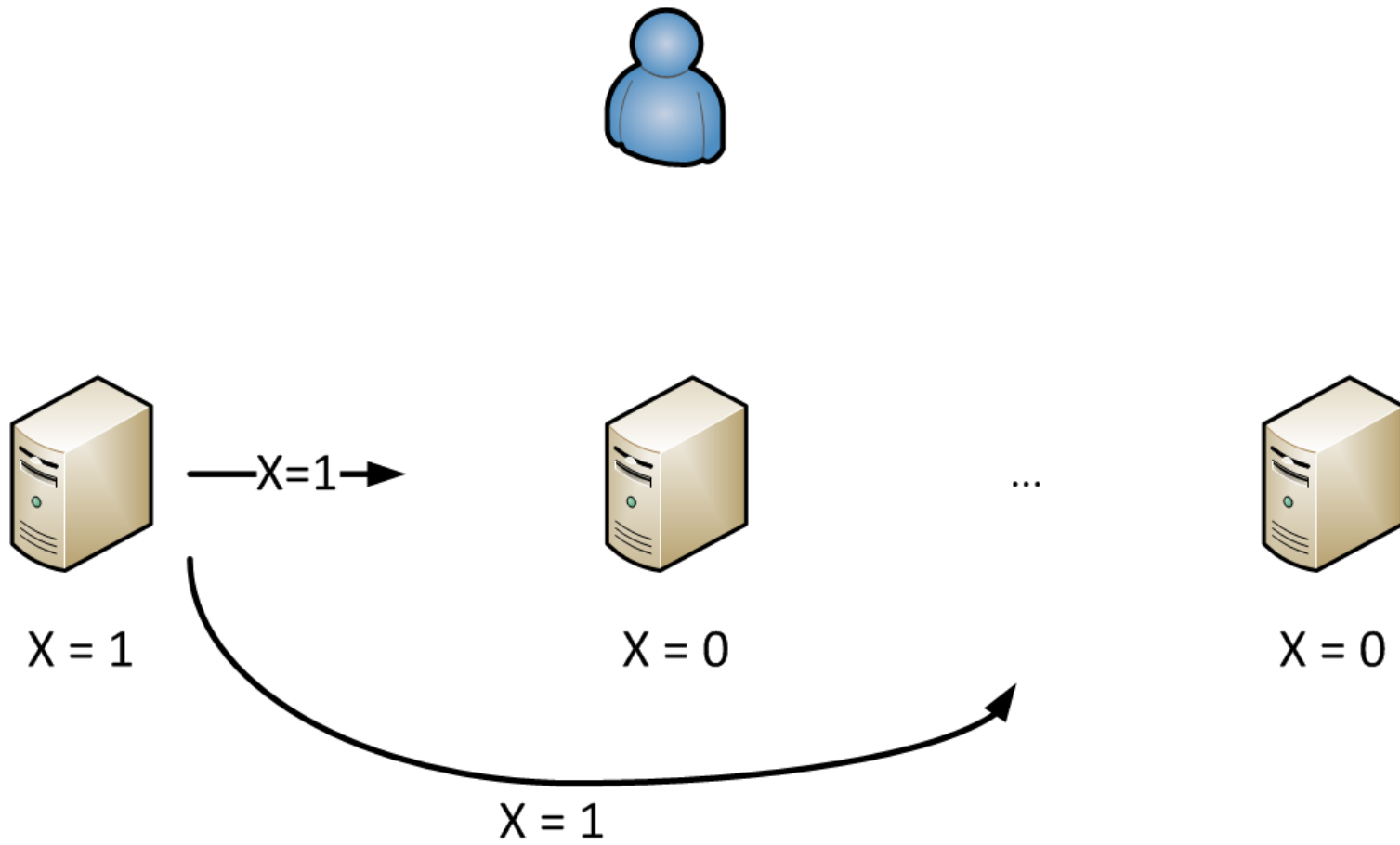
$X = 0$

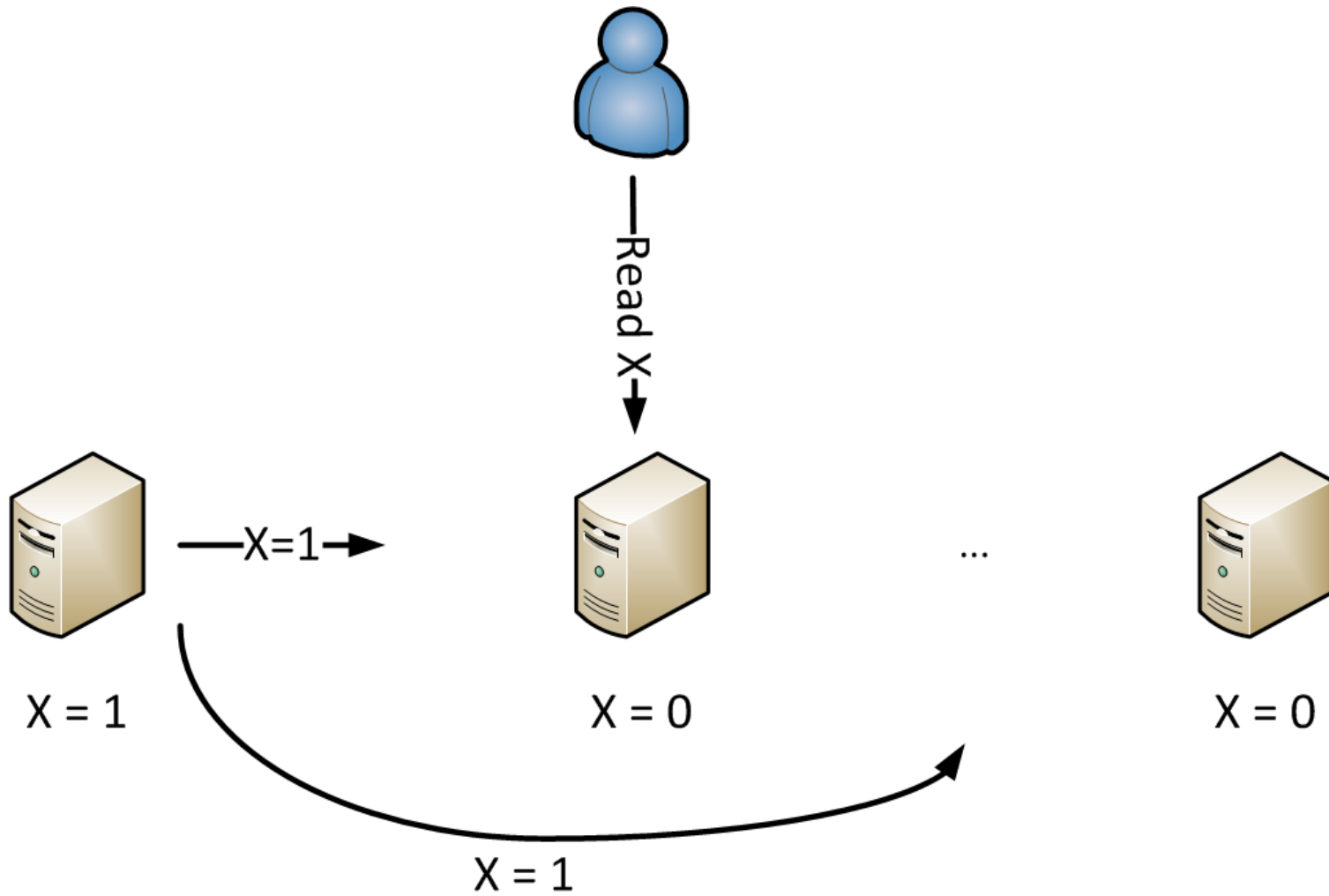
...

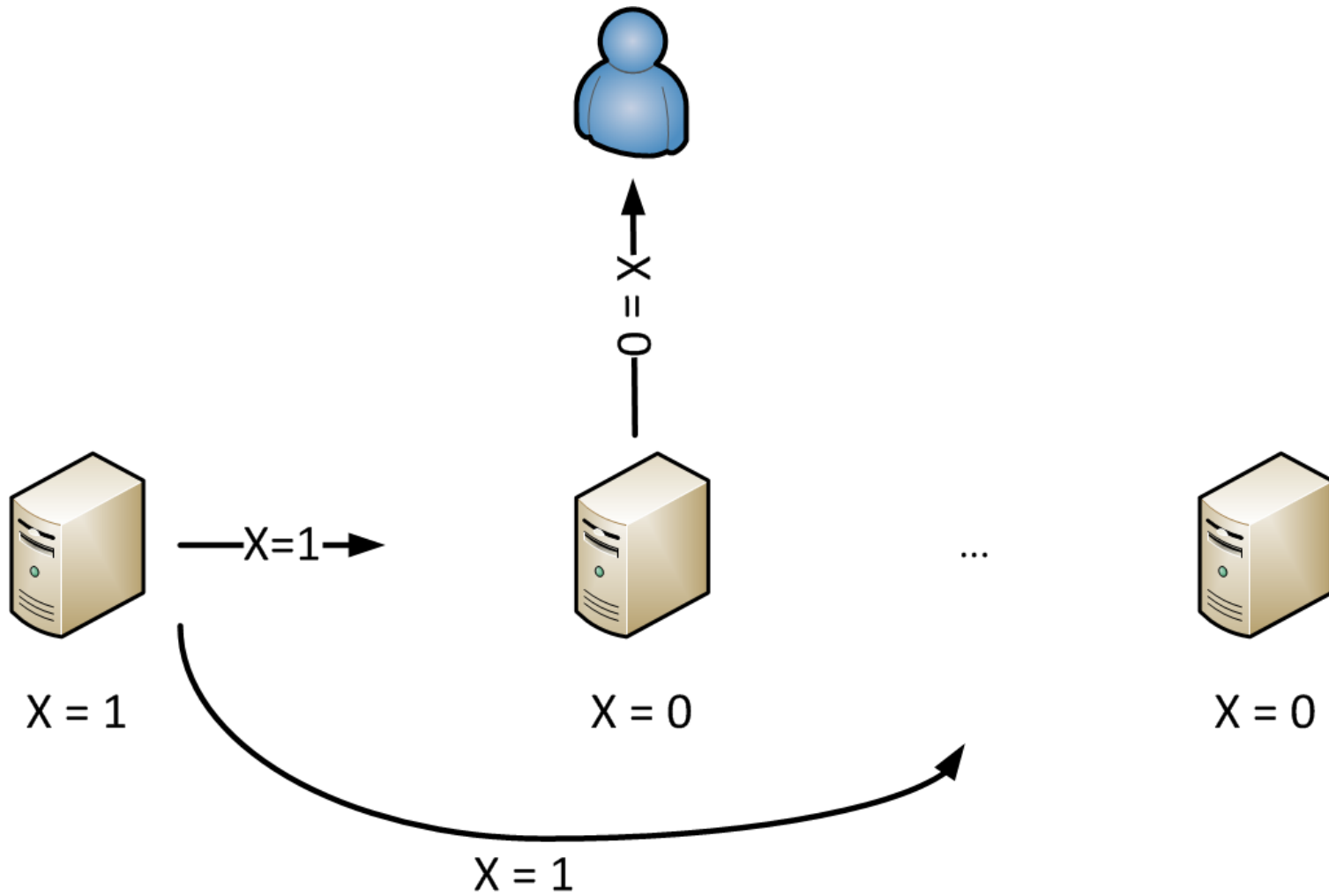


$X = 0$









Consistency model

- A consistency model defines the semantics of read operation of a data store
- That is, the set of possible values that can be returned by the data store.

- **Strong Consistency**: clients of a data storage always see the latest value that was written to a data object
- **Eventual Consistency**:
 - The data returned by a read operation is the value of the object at some past point in time but not necessarily the latest value.
 - The replicated objects converge towards identical copies in the absence of updates.

Enforcing Strong Consistency (Quorum algorithm)

- Two weights are given.
 - One is for the write operation ww write wait
 - The other is for the read operation rw read wait
 - Each replication site has one **token** that carries some weight (normally 1). 令牌
- In order to carry out a read/write operation, a client needs to collect enough weight (i.e. tokens).
 - A read operation can be carried out if the weight collected by the client is greater or equal than rw ;
 - A write operation can be carried out if the weight collected by the client is greater or equal to ww .

$ns=10$
 $ww=6$
 $rw=5$ → needs 5 to read, but only 4 remain
total number of tokens

- rw and ww have to satisfy the following:

- $ww > ns/2$

- ensure that the write operations are exclusive

- $(rw + ww) > ns$ read and write can not happen

- ensure that read and writes excludes each other the same time
- a read operation will never read a value which is out-of-date

- ns is the number of tokens in the system, and each token carries weight 1

允许同时读

不许同时读与写

Enforcing Eventual Consistency

- Enforcing eventual consistency relies on two properties:
 - *total propagation*: updates can reach all the sites
 - *consistent ordering*: updates are processed by all the sites in consistent order

Order is not always important

increase 10

increase 20

$X=1$

$X=1$

Issues with Strong and Eventual Consistency

- **Strong** consistency generally results in **lower performance** and reduced availability for reads or writes or both.
- **Eventual** consistency can **confuse** users and applications
 - A user may write a value to a data item and then later reads an older value.
 - A user may update some data item based on reading some other data, while others read the updated item without seeing the data on which it is based.
 - These issues can appear even when only a single user or application is making data modifications.

Consistency models in industry

- Microsoft Azure: strong consistency
- Amazon S3: eventual consistency
- Many others: between strong and eventual

CAP Theorem for distributed systems

- Cloud providers normally **replicate** data at different locations.
 - For availability and reliability
- To maintain the consistency of the replicated data, the operations on the replicated data need to be synchronised.
- The CAP theorem states that any networked shared-data system can have at most two of three desirable properties:
 - **consistency** (C) equivalent to having a single up-to-date copy of the data;
 - high **availability** (A) of that data (for updates);
 - tolerance to network **partitions** (P).

even if crashed

can provide same

service

分区

系统分开，但
仍在工作

Consistency

- Consistency means the multiple copies need to be kept perfectly **synchronised**.
 - To require perfect consistency, **communication** between data centers hosting the copies is paramount.
 - The overall performance of such a system could drop as the number of replica required goes up.

Availability

- Although having a single copy of a data item ensures perfect consistency, such an arrangement does not provide high availability.
 - The copy becomes unavailable when the site hosting the copy goes down
- The only solution to the availability issue is through replicating the data.
- Increasing the number of sites with copies of the data directly increases the availability of the system.
- Replicas also help in load balancing concurrent read operations.

Partition Tolerance

- We get **Availability** by **replicating** data on different data centers.
- If the network connectivity between two data centers is lost, both data centers are incapable of synchronizing state with each other.
 - If we allow **read/write** operations on these two data centers, it can be shown that the data in the two data centers won't be consistent anymore.
 - If we decide consistency is important, and **disable write** operations during the network outage, we will lose "availability" as no update operations can be carried out.

- Many applications require high **availability** and good **performance**.
- According to the CAP theorem, there is a **trade-off** between consistency and availability.
- In the presence of partition, many systems **choose availability** at the expense of consistency (temporarily).
 - The consistency will be restored after the partition is repaired.
 - Eventual consistency
- The **NoSQL** movement is about creating choices that focus on availability first and consistency second. *not only SQL*
- NoSQL systems use **BASE** (basically available, soft state, eventually consistent).

Basically available

- The system supports data **availability** in a partial system failure.
 - there will be a **response** to any request.
 - some responses might **fail** to obtain the requested data or the data may be in an **inconsistent** state
- For example, if data are partitioned across several servers, BASE design encourages crafting operations in such a way that a server failure **impacts** only the operations accessing the data on the failed server.

Soft State

- For many operations, it **takes some time** for the states of the system to settle.
 - Transfer money between two accounts
 - After deducting an amount from one user, the amount to be credited to the other user is placed in a message queue waiting to be executed
 - There is a lag between the start and the end of the transfer when the money is in the message queue and does not appear in neither users' accounts
- For many applications, this time lag is **acceptable**.
- Accepting the state of the system is constantly changing means accepting the fact that sometimes the data returns from the system might **not be accurate**.

Eventually Consistent

- The system will **eventually** become consistent once it stops receiving input.
 - Updates will be propagated to every site eventually

To implement BASE, many systems rely on **transaction** and some sort of **message queue** to *reliable* *software* *send* *message* persistently store and route data to various storage services that perform the actual database operations.

Begin transaction *local machine.*
Insert into sell_record(id, seller_id, buyer_id, amount);
Queue message *remote* "update user("seller", seller_id, amount)" }
Queue message "update user("buyer", buyer_id, amount)" }
End transaction
ensure send to remote machine.

- In BASE conforming systems, concurrent and possible **inconsistent** updates might occur.
 - Concurrent updating the same calendar entry while the system is partitioned.
- Partially ordered timestamps and event logging should be used for the events in a BASE conforming system.
 - Identifying and resolving conflicting and inconsistent updates.

reviews

- Understand strong consistency and eventual consistency.
- How to ensure strong consistency?
- What are the issues with strong consistency and eventual consistency?
- Understand the CAP theorem for distributed systems.
- Assume you are developing a banking and a shopping cart application. Discuss how the CAP theorem and the operating environment might affect your development.
- What are the advantages of a system that prefers CP? What are the challenges facing the system?
- What are the advantages of a system that prefers AP? What are the challenges facing the system?

reviews

- Understand the BASE properties.
- Why do many systems conform to the BASE properties.
- In BASE, how does a system ensure that an operation can be carried out in the presence of a system failure? [Note: a failure could be a crash failure of a site or a system partition]
- If you design a system conforming to the BASE properties, what issues do you need to address in the presence of system partition?

Further readings

- Douglas B. Terry, Alan J. Demers, Karin Petersen, Mike Spreitzer, Marvin Theimer, and Brent W. Welch. 1994. Session Guarantees for Weakly Consistent Replicated Data. In *Proceedings of the Third International Conference on Parallel and Distributed Information Systems (PDIS '94)*. IEEE Computer Society, Washington, DC, USA, 140-149.
- Eric Brewer, "CAP Twelve Years Later: How the "Rules" Have Changed," *Computer*, vol. 45, no. 2, pp. 23-29, Feb. 2012
- Dan Pritchett. 2008. BASE: An Acid Alternative. *Queue* 6, 3 (May 2008), 48-55.